

Operator overloading

Implement class `Item` for storing information of online grocery store items.

The class must use `std::string` for storing the name of the item, an `int` for the weight of the item in grams, a `double` for price of the item in euros, and an `int` for the number of items available. All data member must be private.

Class must define a constructor that takes the item information as parameters and there must be a default constructor.

The class must have the following operators:

- Output operator for printing the item information using JSON-format. **No newlines may be printed by the operator.**
 - For example:

```
{ "Name":"Diced chicken", "Weight":350, "Price":5.30 }
```
- Input operator for reading the housing information from a stream. **The operator must accept the format produced by the output operator.** The name of the item may contain spaces so the operator must be able to read a name with spaces in it.
- Less than operator that compares two items by price/kg.

Class must define the following member functions:

- `getCount` for getting the current item count
- `setCount` for setting the number of available items
- `getPrice` for getting the price of the item
- `setPrice` for setting the price of the item

Implement a program that defines a vector of `Items`.

The program asks user to enter item information, starting with the name or “stop” to end entering items or “json” to enter item information in JSON format. Each entered item is added to the vector. For testing create a text file with couple of different JSON formatted item to copy-paste as input.

When user has entered all items, the program sorts the vector using `std::sort` and prints the item information. Each item is printed on a separate line with an index number. The net worth of the stock is printed after the list.

Then program asks user to choose item to change by entering the index number and asks user to enter a new price for the item. Then the program sorts and prints the vector again as described earlier.

Notes:

To read a string from a stream using other than default delimiter (white space) you can use `std::getline`. It has overload that takes three parameters: stream to read from, reference to a string, and delimiting character. For example:

```
std::getline(std::cin, my_str, ':');
```